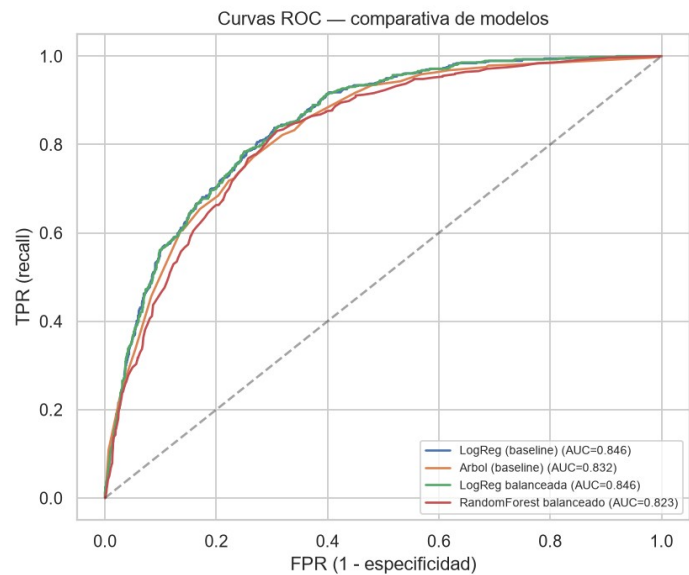


Informe Técnico — Evaluación Parcial N°3

Modelo de IA para la Predicción de Churn: entrenamiento, rendimiento, seguridad e integración BI



Asignatura	ITY1101 — Gestión de Datos para IA
Sección	003V
Caso	Telco Customer Churn (clasificación de abandono)
Integrantes	Benjamín Heresmann — Ingeniero de Datos Diego Hernández — Ingeniero de BD / DataOps
Metodología	PMBOK híbrido (predictivo + adaptativo)
Entrega	Junio 2026

Índice

1. Resumen ejecutivo
2. Fase 1: pipeline DataOps (Parcial 2) y mejoras incorporadas
3. Entrenamiento del modelo de IA
 - 3.1 Diseño del modelo, calidad y partición de datos
 - 3.2 Análisis univariado y bivariado
 - 3.3 Preprocesamiento
 - 3.4 Modelos, entrenamiento y justificación
 - 3.5 Métricas e interpretación
4. Análisis de rendimiento en la nube
5. Auditoría de seguridad y Ley 21.719
6. Integración con Business Intelligence (dashboard)
7. Limitaciones y propuestas de mejora
8. Conclusiones
- Anexo A — Evidencias y recursos

1. Resumen ejecutivo

Este informe documenta la **Evaluación 3** del proyecto Telco Customer Churn, que construye sobre el pipeline DataOps de la Evaluación 2 para **entrenar un modelo de inteligencia artificial supervisado** que predice el abandono de clientes (*churn*). A partir de los 7.043 clientes ya curados y almacenados en la base de datos relacional, se entrena un clasificador binario, se mide su rendimiento en la nube, se audita la seguridad del sistema frente a la nueva normativa chilena de datos personales, y se integran los resultados en un **dashboard de Business Intelligence** interactivo.

El modelo seleccionado —**Regresión Logística balanceada**— alcanza un **recall del 79.7%** sobre el conjunto de prueba: detecta cuatro de cada cinco clientes que efectivamente abandonan, la métrica crítica para una estrategia de retención, donde el costo de no detectar a un cliente que se va (falso negativo) supera al de una falsa alarma. El sistema completo —pipeline, modelo, base de datos y dashboard— opera sobre una **arquitectura cloud desacoplada** (GitHub · Railway · Supabase) y se construyó desde su diseño para cumplir la **Ley 21.719** de protección de datos personales (*privacy by design*).

2. Fase 1: pipeline DataOps (Parcial 2) y mejoras

La Fase 1 entregó un **pipeline DataOps de cuatro etapas** —ingesta, limpieza y transformación, validación estructural y semántica, y carga— que deja 7.043 registros limpios en **PostgreSQL 17 (Supabase)**, listos como fuente de entrenamiento. El procesamiento se expone como **API REST en Railway** (FastAPI, documentada en Swagger) y el código se versiona en **GitHub** con integración continua (pytest en cada push).

Mejoras incorporadas tras la retroalimentación del docente: (1) la solución dejó de ser un monolito y se desacopló en servicios cloud independientes; (2) cada etapa del pipeline se **modulariza como un contenedor por capa** (docker-compose.yml: ingesta, limpieza, validación, carga), de modo que cada capa se modifica y escala por separado, con la salida de cada zona destinada a Supabase Storage en la arquitectura objetivo; (3) se habilitó **Row-Level Security** en todas las tablas y se enriquecieron las respuestas de la API con métricas por etapa. Sobre esa base sólida se levanta la Fase 2 (este informe).

3. Entrenamiento del modelo de IA

3.1 Diseño del modelo, calidad y partición de datos

Tipo de problema: clasificación binaria supervisada. La **variable objetivo** es churn (Sí/No); las **variables predictoras** son los atributos del cliente (antigüedad, tipo de contrato, cargos, servicios, método de pago, etc.). Se elige el paradigma supervisado porque el dataset histórico ya contiene la respuesta correcta (churn) para cada cliente.

Calidad de datos: el dataset proviene del pipeline validado de la Fase 1: 7.043 filas, **0 valores nulos**, tipos correctos y reglas de negocio verificadas. La clase está **desbalanceada**: solo el **26,5%** de los clientes abandona (1.869 de 7.043), un factor decisivo en la elección de métricas y del manejo del desbalance.

Métrica prioritaria: se prioriza el **recall** (sensibilidad) sobre la exactitud, porque en retención el error más costoso es el **falso negativo** —un cliente que se va y el modelo no detecta— frente al falso positivo —una falsa alarma que solo gasta un incentivo de retención—.

Partición: división **70/30 estratificada** (stratify), que mantiene la proporción de churn en ambos conjuntos: **train = 4.930** (26,5% churn) y **test = 2.113** (26,5% churn). La estratificación es esencial con clases desbalanceadas para que la evaluación sea representativa.

3.2 Análisis univariado y bivariado

El análisis **univariado** de las variables numéricas muestra una antigüedad (tenure) media de 32,4 meses (rango 0–72), un cargo mensual medio de \$64,76 y un cargo total medio de \$2.279,7. El análisis **bivariado** (tasa de churn por segmento) revela los factores de riesgo con claridad:

Segmento	Categoría de mayor churn	Categoría de menor churn
Tipo de contrato	Month-to-month: 42,7%	Two year: 2,8%
Servicio de internet	Fibra óptica: 41,9%	Sin internet: 7,4%
Método de pago	Electronic check: 45,3%	Credit card: 15,2%
Antigüedad (meses)	0–12: 47,4%	49–72: 9,5%
Adulto mayor	Sí: 41,7%	No: 23,6%

Los clientes que abandonan tienen, en promedio, **menor antigüedad** (18,0 vs 37,6 meses) y **mayor cargo mensual** (74,4 vs 61,3). La matriz de correlación confirma que tenure es la variable numérica más asociada (negativamente) al churn.

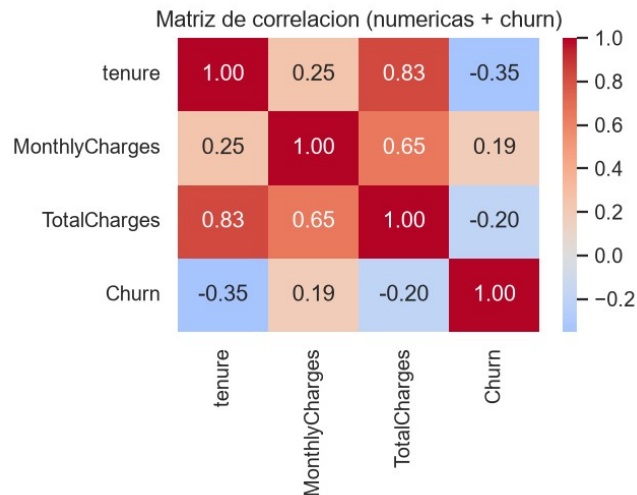


Figura 1. Matriz de correlación de variables numéricas con la variable objetivo.

3.3 Preprocesamiento

Se construye una tubería (Pipeline de scikit-learn) que evita la fuga de datos al ajustarse solo con el conjunto de entrenamiento: **estandarización** (StandardScaler) de las variables numéricas continuas, **paso directo** de las binarias (0/1), y **codificación one-hot** (OneHotEncoder) de las categóricas. El **desbalance** se maneja con `class_weight='balanced'` —que pondera más la clase minoritaria— en lugar de generar datos sintéticos (SMOTE), evitando dependencias extra y el riesgo de distorsión; SMOTE quedó como alternativa evaluada y descartada.

3.4 Modelos, entrenamiento y justificación

Se entrena primero un **modelo base** con los parámetros por defecto que enseña el material — **Regresión Logística** (regularización L2) y **Árbol de decisión** (`max_depth=5`)— y luego una **mejora**: **Random Forest** (`n_estimators=100`) y la **Regresión Logística balanceada**. Todos se evalúan sobre el mismo conjunto de prueba (2.113 clientes):

Modelo	Accuracy	Recall	Precision	F1	Gini
LogReg (baseline)	81.0%	56.0%	67.1%	0.610	0.693
Arbol (baseline)	79.5%	60.6%	61.6%	0.611	0.664
LogReg balanceada	74.2%	79.7%	50.9%	0.621	0.693
RandomForest balanceado	77.1%	63.5%	56.1%	0.596	0.645

Justificación de la elección: se selecciona la **Regresión Logística balanceada** por su **mejor F1 (0.621) y el mayor recall (79.7%)**, coherente con la prioridad de detectar abandonos. Aunque

su exactitud (74,2%) es menor que la del modelo base (81,0%), esa exactitud es **engañoso** por el desbalance: un modelo trivial que predijera “nadie se va” alcanzaría ~73,5% de exactitud sin detectar un solo abandono. Además es un modelo **interpretable**, lo que facilita explicar los factores de riesgo al negocio.

Control de sobreajuste (overfitting): el Random Forest por defecto **memoriza** el entrenamiento (recall de entrenamiento = 100%, exactitud 99,2%) pero cae a 63,5% de recall en prueba — brecha de +0,22—, un caso de libro de sobreajuste que motiva regularizarlo (limitar profundidad) como mejora futura. La Regresión Logística, en cambio, rinde casi igual en train y test (sin sobreajuste).

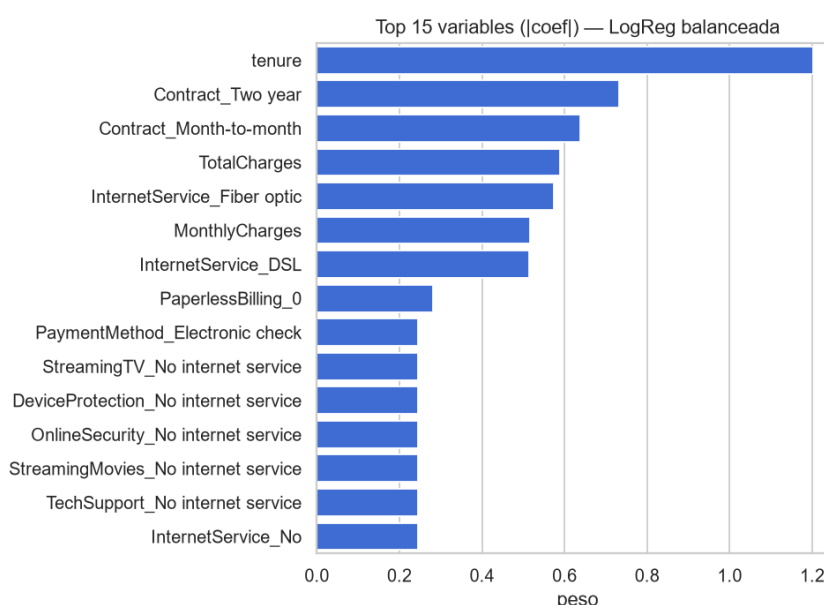


Figura 2. Variables de mayor peso: antigüedad, tipo de contrato, cargos y tipo de internet.

3.5 Métricas e interpretación

La **matriz de confusión** del modelo elegido resume su comportamiento sobre los 2.113 clientes de prueba: de los 561 que realmente abandonan, detecta **447** (verdaderos positivos) y solo deja escapar **114** (falsos negativos); a cambio, genera **431** falsas alarmas (falsos positivos).

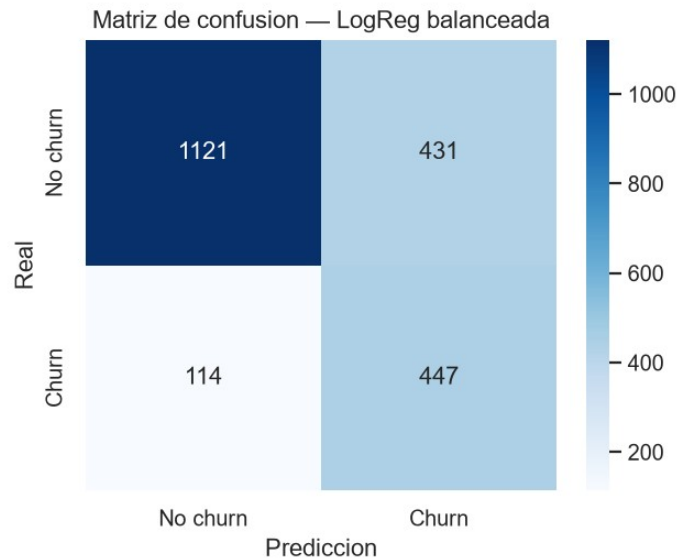


Figura 3. Matriz de confusión — Regresión Logística balanceada (conjunto de prueba).

Interpretación de las métricas: el **recall 79.7%** indica que se capturan 4 de cada 5 abandonos; la **precisión 50.9%** refleja el costo asumido en falsas alarmas (aceptable en retención); el **F1 0.621** equilibra ambos. La **curva ROC** (Figura de portada) y el **coeficiente de Gini de 0.693** (Gini = $2 \cdot \text{AUC} - 1$, con $\text{AUC} = 0.846$) miden el poder discriminante global del modelo: un Gini cercano a 0,7 indica una capacidad de ordenamiento sólida, muy por encima del azar (Gini = 0).

4. Análisis de rendimiento en la nube

Se instrumenta el flujo con `time.time()` y `psutil` para medir el costo de cada operación sobre los 7.043 registros, comparando la **lectura local (CSV)** con la **lectura en la nube (Supabase)** sobre el mismo dato:

Operación	Tiempo (s)	Observación
Lectura local (CSV)	0.018	Acceso a disco, casi instantáneo
Lectura nube (Supabase)	3.939	Latencia de red (~218.8× más lenta)
Entrenamiento del modelo	1.336	Cómputo del ajuste (fit)
Total	5.293	RAM 200.1 MB · CPU 9.9%

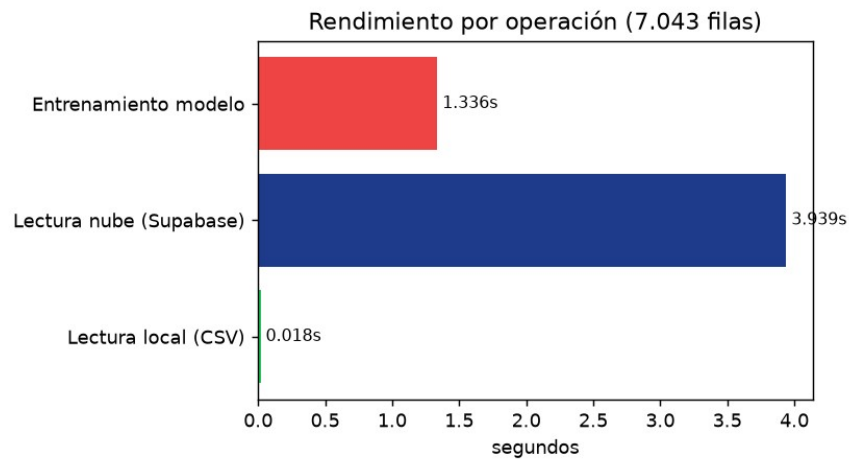


Figura 4. Tiempo por operación. El cuello de botella es la latencia de red hacia la nube.

Hallazgo: el **cuello de botella es la latencia de red** hacia la nube (3,94 s frente a 0,018 s en local, ~219×), no el cómputo —el entrenamiento toma solo 1,34 s—. El consumo de recursos (RAM ≈ 200 MB, CPU ≈ 10%, total ≈ 5,3 s) se mantiene **dentro de los umbrales sanos** para este volumen (referencia del material: 5–15 s, CPU < 50%, RAM < 800 MB). La mejora natural es acercar el cómputo a los datos o cachear la lectura.

5. Auditoría de seguridad y Ley 21.719

La auditoría revisa cuatro frentes con evidencia reproducible. **(1) Credenciales:** la revisión del código y del historial de git confirma **0 secretos** (las claves viven solo en variables de entorno; .env está en .gitignore y nunca fue commiteado). **(2) Accesos:** todas las tablas tienen **Row-Level Security activo sin políticas permisivas** (cerradas por defecto); los roles públicos anon/authenticated no pueden leer, y solo el rol dueño del backend accede. **(3) Entorno:** el escaneo pip-audit detectó 10 CVEs en 3 dependencias (con plan de actualización); se propone trivy para escanear la imagen de Railway. **(4) Logs:** se monitorean accesos fallidos repetidos como posible fuerza bruta.

Privilegio mínimo: se define un rol de **solo lectura** (telco_lectura, en sql/02_roles_seguridad.sql) para que la analítica y el dashboard consulten sin permisos de escritura, separando el rol del pipeline (dueño) del rol de consumo.

Marco legal — Ley 19.628 modernizada por la Ley 21.719 (vigencia plena 01-12-2026, nivel GDPR): el dataset contiene **datos personales** (género, rango etario, situación familiar y datos económicos del cliente) aunque **no datos sensibles** en sentido estricto. El sistema se diseñó para cumplir la nueva ley desde el inicio (*compliance by design*):

Principio (Ley 21.719 / GDPR)	Control técnico implementado
Finalidad y proporcionalidad	Datos usados solo para predecir churn; sin recolección extra
Minimización	customerID es un identificador, no nombre/RUT; solo variables necesarias
Seguridad y confidencialidad	RLS cerrado, privilegio mínimo, TLS en tránsito, secretos fuera del código
Calidad / integridad	Validación + transacciones + auditoría en carga_logs
Responsabilidad proactiva	Controles embebidos desde el diseño; esta auditoría como evidencia
Derechos ARCO	Indexación por customer_id permite rectificar/eliminar a un titular

6. Integración con Business Intelligence

Los resultados del modelo se persisten en una tabla ``predicciones`` de Supabase (probabilidad y clase predicha, acierto y modelo por cliente), que actúa como **fente única** para un **dashboard interactivo** construido con **Streamlit + Plotly**. La integración es de extremo a extremo y automática: `clientes` → `modelo` → `predicciones` → `dashboard`, sin exportaciones manuales.

El panel presenta: **KPIs** del modelo (recall, F1, precisión, exactitud, Gini y clientes en riesgo); la **matriz de confusión** interactiva; la **tasa de churn por segmento**; una **tabla filtrable de errores** para el análisis caso a caso; y el **embudo de volumen por etapa** del pipeline. Se eligió Streamlit por ser **versionable** y **demostrable en vivo** como aplicación web. Las métricas mostradas provienen del conjunto de prueba (holdout), mientras que el scoring de riesgo se calcula sobre toda la base.

Modelo servido como microservicios (entrenamiento e inferencia separados). El modelo no se ejecuta dentro del dashboard: se entrena y se sirve en **dos servicios independientes** que se comunican solo a través de Supabase. Un servicio **trainer** (POST `/train`) entrena el modelo y lo guarda serializado en la tabla `modelo_artefacto`; un servicio **predictor** (`/metrics`, `/predict/cliente/{id}`, `/predict/batch`) lo **carga y predice sin re-entrenar**. Ambos usan la misma imagen y se diferencian solo por una variable de entorno, extendiendo a la capa de IA el principio de **un contenedor por responsabilidad** aplicado al pipeline en la Fase 1.

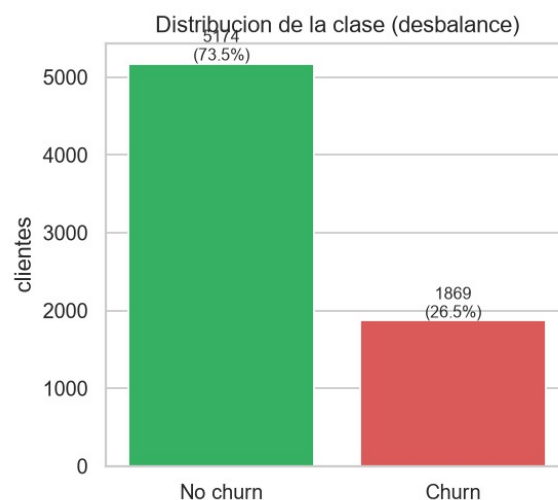


Figura 5. Distribución de la clase (desbalance) — uno de los componentes del dashboard.

7. Limitaciones y propuestas de mejora

Se distinguen **limitaciones** (límites técnicos/operativos conocidos, no fallos) de las **mejoras** derivadas de ellas, cada una con su beneficio y su carácter preventivo o correctivo:

Limitación detectada (con evidencia)	Mejora propuesta	Tipo
Random Forest sobreajusta (train 100% / test 63,5% recall)	Regularizar (max_depth, min_samples_leaf)	Correctiva
Precisión baja del modelo (51%): muchas falsas alarmas	Ajustar el umbral de decisión / rankear por probabilidad	Correctiva
Latencia de red domina (3,9 s nube vs 0,018 s local)	Acercar cómputo a los datos / cachear lecturas	Preventiva
Dataset pequeño y desbalanceado (7.043; 26,5%)	Validación cruzada k-fold; más feature engineering	Preventiva
Free tier se suspende por inactividad	Keep-alive programado o tier pago	Preventiva
10 CVEs en dependencias (starlette/dotenv/pytest)	Actualizar python-dotenv; planear upgrade FastAPI	Correctiva

Implementación priorizada (equipo de 2): primero las de mayor impacto y menor costo sobre lo ya construido —ajuste de umbral y regularización del modelo—, luego las de seguridad (actualización de dependencias y rol de solo lectura), y finalmente las de rendimiento (caché de lecturas). La separación del modelo en microservicios de entrenamiento e inferencia, inicialmente propuesta como mejora, **ya fue implementada y desplegada** (servicios trainer y predictor). Cada mejora es incremental; ninguna exige rehacer el pipeline.

8. Conclusiones

La Evaluación 3 cierra el ciclo de vida del dato: sobre el pipeline DataOps de la Fase 1 se entrenó un **modelo de IA supervisado** que predice el churn con un **recall del 79.7%**, se midió su **rendimiento en la nube**, se **auditó la seguridad** del sistema frente a la Ley 21.719 y se **integraron los resultados en un dashboard BI** interactivo. La solución es coherente de extremo a extremo, reproducible y construida con criterios de privacidad desde el diseño.

El principal aprendizaje técnico es que, en un problema **desbalanceado** como el churn, la **exactitud engaña** y la elección de la métrica (recall) y del manejo del desbalance (class_weight) determina el valor real del modelo para el negocio. Las mejoras propuestas —ajuste de umbral, regularización y endurecimiento de seguridad— trazan un camino claro y realista para la siguiente iteración.

Anexo A — Evidencias y recursos

Recurso	Ubicación
Repositorio GitHub	github.com/BenjaminHeresmann/telco-churn-pipeline
API en producción (Swagger)	telco-api-production-e466.up.railway.app/docs
Base de datos	PostgreSQL 17 en Supabase (proyecto telco-churn)
Modelo y métricas	<code>src/modelo.py</code> · <code>outputs/modelo/</code>
Auditoría de seguridad	<code>outputs/seguridad/auditoria_seguridad.md</code> · <code>sql/02_roles_seguridad.sql</code>
Rendimiento	<code>src/benchmark.py</code> · <code>outputs/rendimiento/</code>
Dashboard BI	<code>dashboard/app.py</code> (<code>streamlit run dashboard/app.py</code>)

Las cifras de este informe se generan automáticamente desde los artefactos reales del proyecto (`outputs/modelo/metricas_modelos.csv` y `outputs/rendimiento/benchmark.json`), garantizando su trazabilidad y reproducibilidad.